Deterministic methods to find primes

Terence Tao · 21 September 2010

https://terrytao.wordpress.com/2010/09/21/a-deterministic-way-to-find-primes/

tao2typ · converted to Typst 2026-10-14 (UTC)

I've uploaded to the arXiv the polymath research paper "Deterministic methods to find primes", which is the outcome of the Polymath4 collaborative mathematics project, and has been submitted to Mathematics of Computation.

The objective of this paper was to find fast *deterministic* algorithms to solve the following problem:

> Given a (large) integer x, find a prime p larger than x.

Thanks to the AKS algorithm, a number of size O(x) can be deterministically tested for primality in time $O(\log^{O(1)} x) = O(x^{o(1)})$. By Bertrand's postulate, there is always at least one prime between x and 2x; by testing each one of these integers in turn for primality, one can thus obtain a deterministic algorithm to find primes in time $O(x^{1+o(1)})$.

But one should be able to do much better. For comparison, if probabilistic algorithms are allowed, then by randomly selecting integers between x and 2x to test for primality, it is easy to see from the prime number theorem that one will succeed in obtaining a prime with high probability in time $O(\log^{O(1)} x) = O(x^{o(1)})$. However, after some effort we were not able to "derandomise" this algorithm to create any reasonable deterministic counterpart. Nevertheless, we conjecture that a deterministic algorithm with run time $O(x^{o(1)})$ exists. Such algorithms can be easily obtained if one assumes some standard conjectures regarding the primes (e.g. Cramer's conjecture on prime gaps), but we do not know of any deterministic algorithms which can be unconditionally proved to run in time $O(x^{o(1)})$.

Currently, the best known deterministic algorithm is due to Lagarias and Odlyzko, and has a run time of $O(x^{1/2+o(1)})$. Roughly speaking, it is based on the ability to compute the prime counting function $\pi(x)$ in time $O(x^{1/2+o(1)})$; once one has this function, one can detect which intervals contain primes or not, and then starting from Bertrand's postulate and performing a binary search one can then locate a prime. The Lagarias-Odlyzko argument is based on approximating $\pi(x)$ by a certain integral of the Riemann zeta function, which one then approximates in turn by quadrature.

We conjecture that one should be able to compute $\pi(x)$ in faster time, and in particular in time $O(x^{1/2-c+o(1)})$ for some $c > 0$. Unfortunately, we were not able to achieve this; however, we do have a non-trivial method to compute the *parity* $\pi(x) \mod 2$ of $\pi(x)$ in such a time; a bit more generally (and oversimplifying a little bit), we can compute various projections $\sum_{p \leq x} t^p \mod 2, g(t)$ of the prime polynomial $\sum_{p \leq x} t^p$ modulo some small polynomials g. This seems close to being able to achieve the goal of detecting whether primes exist in a given interval, but unfortunately some additional work seems to be needed in that area. Still, the methods used to get the parity seem to be interesting (involving the Dirichlet hyperbola method, a piecewise approximation of the hyperbola, and the Strassen fast multiplication algorithm), and these techniques might be useful for some other problems.

Roughly speaking, the idea to compute the parity of $\pi(x) \mod 2$ is as follows. The first observation is that, for square-free n, the number $\tau(n)$ of divisors of n is equal to 2 when n is a prime, and a multiple of 4 otherwise. So to compute the parity of $\pi(x)$, it suffices to compute $\sum_{n \leq x} \tau(n)$ modulo 4 (or more precisely, the restriction of this sum to squarefree n).

The restriction to squarefree numbers turns out to be relatively easy to handle, so we ignore it. Since $\tau(n) = \sum_{a,b:ab=n} 1$, we can rewrite

$$\sum_{n \leq x} \tau(n) = \sum_{a,b:ab \leq x} 1.$$

So it suffices to find an efficient way to count the number of lattice points below the hyperbola $\{(a,b) : ab = x\}$.

The classic way to do this is via the Dirichlet hyperbola method. This lets one rewrite the above expression as

$$2 \sum_{a \leq \sqrt{x}} \left\lfloor \frac{x}{a} \right\rfloor - \lfloor \sqrt{x} \rfloor^2$$

(assuming x is not a perfect square, where $\lfloor x \rfloor$ is the greatest integer function). One can then compute this quantity in time $O(x^{1/2+o(1)})$ without difficulty.

To improve this to $O(x^{1/2-c+o(1)})$, we used some ideas of Vinogradov, based on using Diophantine approximation to find moderately long arithmetic progressions on which the map $a \mapsto \left\lfloor \frac{x}{a} \right\rfloor$ is linear, and thus easily summable.

The same method extends to the polynomial setting, though now, instead of summing an arithmetic progression, one has to find an efficient way to sum quadratic sums such as $\sum_{n=1}^{N} t^{an^2+bn+c}$. This turns out to be possible by expressing this sum as a matrix product and then using fast matrix multiplication algorithms.

There is still some scope to improve the results and methods further; Ernie Croot is pursuing this with two undergraduate students, David Hollis and David Lowry,

Comments (15)

 **Kristal Cantwell** · 21 September 2010

When I look at the paper in the link the title is “Deterministic methods to find primes” instead of “A deterministic way to find primes”. There seems to be a dscrepancy in titles here.
[Title corrected, thanks. -T]

 **Ryan Williams** · 22 September 2010

Hi, this is very interesting work. Wish I’d seen the project earlier.
Concerning Lemma 3.1: Schoenhage-Strassen is a reference for fast integer multiplication, not matrix multiplication.
It would be surprising to me if the expression in Lemma 3.1 cannot be solved using a Fast Fourier Transform instead of matrix multiplication (in the most optimistic case, this would result in a complexity of q^{1/2} polylog q rather than q^{1-c}). These matrices are extremely structured. But I understand you are just going for any improvement at all…

 **Terence Tao** · 22 September 2010

Oops, thanks for pointing that out! I’ve changed the reference, which should appear in the next arXiv version.

 **Anonymous** · 28 September 2010

Wow, you are still trying to use 19th century mathematics to solve a 21st century problem? Just wow.

 **.** · 28 September 2010

Katharine has a point. Very few people have knowledge of sophisticated techniques in mathematics and probably those people are too busy to join polymath projects. Hence may be polymath approach may not work at the highest realm of mathematics.

 **Terence Tao** · 28 September 2010

Well, one of the great things about mathematics is that it is cumulative; a tool developed centuries ago can still be used today. For instance, in my work with Ben Green (and Tamar Ziegler) on establishing linear patterns in primes (the final component of which was only finished this month), at one point we relied crucially on a 2300 year old lemma of Archimedes.
In this particular case, we did consider using some quite high powered machinery, but strangely enough it ended up giving worse results than a more “low-tech” approach. For instance, using the Friedlander-Iwaniec theorem (perhaps one of the deepest results in sieve theory) one only obtained an algorithm with a run time of $O(x^{3/4+o(1)})$ (or $O(x^{2/3+o(1)})$ if one uses Heath-Brown’s variant of that theorem), as opposed to the Lagarias-Odlyzko $O(x^{1/2+o(1)})$ algorithm (which came out in 1987, but uses fairly classical methods that would already have been available in the 1920s). In general, highly advanced machinery is best suited for problems in mature, well-developed fields. One of the issues here was that while the question was quite simple to state, there had not really been a substantial body of literature built up around this and similar problems. In those situations, classical methods are often quite competitive.

 **.** · 28 September 2010

Professor Tao: Your point is also very correct. Most modern machinery are built from very vast abstraction of classical techniques having been built for specific purposes or problems at hand. At the core one probably can get quite good intuition even with classical methods and it is true that not all classical techniques are put in modern machinery because the need may not have arisen.

Hence it may be quite wrong to discard a technique as classical without understanding the fact that a shaving razor may do when a laser may not. But polymath may not attract people from quite mature mathematical fields.

 **Eric** · 2 October 2010

It seems to be possible to multiply the matrices of Lemma 3.1 in time $O(N^{o(1)}Q^2)$, up to logarithmic factors in Q (everything is "up to log factors in Q"; I won't mention it all the time).
Consider a matrix M with entries $t^{ai^2+bj^2+cij+di+ej+f}$, with $a, ..., f = O(N)$. The main claim is that one can multiply this matrix by a vector in time $O(N^{o(1)}Q)$.
One can factor M as $D_1 M' D_2$, with D_1 a diagonal matrix with entries t^{ai^2+di+f}, and D_2 a diagonal matrix with entries t^{bj^2+ej}. Multiplication of D_1 or D_2 by a vector is done in time $O(N^{o(1)}Q)$.
The matrix M' has entries t^{cij}. To do the matrix-vector product by M', one uses the chirp transform algorithm; this takes time $O(N^{o(1)}Q)$, using FFT polynomial multiplication. The chirp transform algorithm actually involves divisions by power of t; I'm not sure if it's a problem in your context (and I don't know how to get rid of them...).

 **ahmed** · 23 September 2011

I was doing some math homework with my 10 year and the general question about prime numbers came up. We did some general explorations to try and find relevance to help learning and we came across your lecture at Northwestern. Useful in cryptography was the best I could do on an explanation.
A few weeks later: but that use would become defunct if you could calculate Pn.....
What if Pn could be found by looking at the point at which gravity becomes uneven?
Given an equal number of equally distributed particles, gravity is evenly distributed as every particle is pulling equally on every other particle. If one particle is lost, the equal distribution collapses and gravity becomes uneven causing the clustering of particles whose gravity increases causes further instability.
In this case Pn = ∞-1/∞E
Where:
∞ = Σ of all relevant particles
E = energy of relevant particles

 **Anonymous** · 10 September 2012

I have the following question/idea: By the Dirichlet Theorem we know that the sequence $ak + b$ has infinitely many primes. Assume that this sequence is $(p_1, p_2, ...)$, where $p_i = ak_i + b$. Can we find a bound $k_{i+1} \leq f(a, b, k_i)$? Then knowing p_i, we minimize $f(a, b, k_i)$ with respect subject to $p_i = ak_i + b$ to find its upper bound k^*, yielding $k_{i+1} \in [k_i + 1, k^*]$.

>  **Terence Tao** · 10 September 2012
>
> If I understand your comment correctly, you are hoping to bound the gap between consecutive primes in arithmetic progressions, after rescaling by the progression spacing. The bounds in this problem are comparable to those for bounding the gap between consecutive primes in the integers; roughly speaking, using a progression of spacing a makes the primes denser by a factor of $a/\phi(a)$, but this is a fairly small factor ($a/\phi(a)$ can't get much bigger than $\log \log a$) and is unfortunately does not give much of an asymptotic saving.

 **Ivan Borsenco** · 10 September 2012

Thank you for reply. Yes, that was exactly what I meant. I thought choosing wisely parameters a,b for a given x, we can assure the existence of p in its proximity rescaled by arithmetical progression.

 **Eytan Paldi** · 14 July 2015

Another approach is to use a prime generating polynomial $P(x_1,...,x_n)$ (which has the property that its value is a prime number whenever it is positive for nonnegative integer arguments) several explicit such polynomials are known.

The basic idea (given sufficiently large x) is to locate $X=(x_1,...,x_n)\in[1/2,\infty)^n$ with the property P0 that $P(X')>x$ whenever $X'=(x'_1,...,x'_n)\in\mathbb{R}^n$ satisfies

$|x'_i-x_i|\le 1/2$ for $i=1,...,n$ (*)

Therefore, if such X (with the above property P0) exists, then by (*) there is $X'\in\mathbb{R}^n$ with nonnegative integer components, and therefore $P(X')$ must be a prime larger than x.

To use this approach, let $S_{X,x}$ be the set of $X'\in\mathbb{R}^n$ which satisfy (*) together with $P(X')\le x$. Clearly, the set $S_{X,x}$ (defined for each $x\in\mathbb{R}$ and $X\in\mathbb{R}^n$) is a semialgebraic set of $\mathbb{R}^n$. Moreover, if $S_{X,x}$ is empty, then X should satisfy the above property P0. Hence, to apply this approach, one needs only to locate $X\in[1/2,\infty)^n$ for which $S_{X,x}$ is empty!

Since $S_{X,x}$ is (explicitly constructible) semialgebraic set, it is a finite union of basic semialgebraic sets, and therefore it is empty if and only if $X\in S_x$ where S_x is another (explicitly constructible) semialgebraic set of R^n. Therefore, one needs only to verify that there is $X\in S'_x:=[1/2,\infty)^n\cap S_x$.

Since S'_x (as a semialgebraic set of $\mathbb{R}^n$) is nonempty if and only if $x\in S$ where S is (explicitly constructible) finite union of real intervals, it remains only to verify that S contains every sufficiently large x. This can be done (at least in principle!), and if it is true (i.e. S'_x is nonempty for sufficiently large x), then the complexity depends mainly on finding X in S'_x (using algorithmic real algebraic geometry) since finding X' from X has bounded complexity (just rounding the components of X) and the computation of the prime $P(X')$ has polynomial complexity in the total number of bits representing X').

 **onlyearthleft.** · 3 December 2016

Would it be interesting to prove anything non-trivial like improving the O(\log n) randomized trials needed to get a prime of magnitude n (number of bits is log2 n) assuming abc conjecture?

Or will such a result be no good?

Or do you already know deterministic improvement assuming abc?

 reality · 30 December 2017

The restriction to squarefree numbers turns out to be relatively easy to handle, so we ignore it. Is it sieved away?